Week 3: Camera Mouse Implementation

1. Introduction

The objective of this week's assignment was to extend the skin color detection algorithm from Week 2 to create a real-time "Camera Mouse". By tracking the position of a face or hand using a webcam, we map its movement to the computer's cursor, enabling hands-free control. This involves calculating the centroid of the detected object, mapping coordinates to screen dimensions, and applying smoothing for better usability.

2. Methodology

2.1. Skin Color Detection

We reused the HSV thresholding logic from Week 2. The camera frame is converted to HSV color space, and a binary mask is created using specific ranges for skin tones:

- **Hue**: 0-18 and 162-179 (Red spectrum)
- **Saturation**: 51-153
- **Value**: 102-255

Morphological operations (Opening and Dilation) are applied to remove noise and fill small holes in the mask.

2.2. Centroid Calculation

To track movement, we identify the center of the detected object.

1. **Contours**: We use `cv2.findContours` to find outlines in the binary mask.
2. **Filtering**: We select the largest contour by area to ignore smaller background noise.
3. **Moments**: Using `cv2.moments`, we calculate the spatial moments of the largest contour.
4. **Centroid**: The center (cx, cy) is derived as: $$ cx = \frac{M_{10}}{M_{00}}, \quad cy = \frac{M_{01}}{M_{00}} $$

2.3. Mouse Control and Mapping

The computed centroid coordinates from the camera (typically 640x480 or 1920x1080) must be mapped to the screen resolution (e.g., 1470x956).

- **Mirroring**: The camera feed is flipped horizontally so that moving right physically moves the cursor right/left naturally.

- **Calibration**: A "neutral" position is established when the user presses 'c'. All movements are calculated relative to this origin.
- **Scaling**: A sensitivity factor is applied. `Target = Center + (Delta * Scale)`. This allows the user to reach the edges of the screen with smaller head movements.
- **Smoothing**: To prevent cursor jitter due to camera noise, we use Linear Interpolation (Lerp): $$ x_{new} = x_{prev} + (x_{target} - x_{prev}) \times \alpha $$ Where α is a smoothing factor (e.g., 0.2).

3. Implementation Code

The core logic for tracking and calculation is implemented in Python:

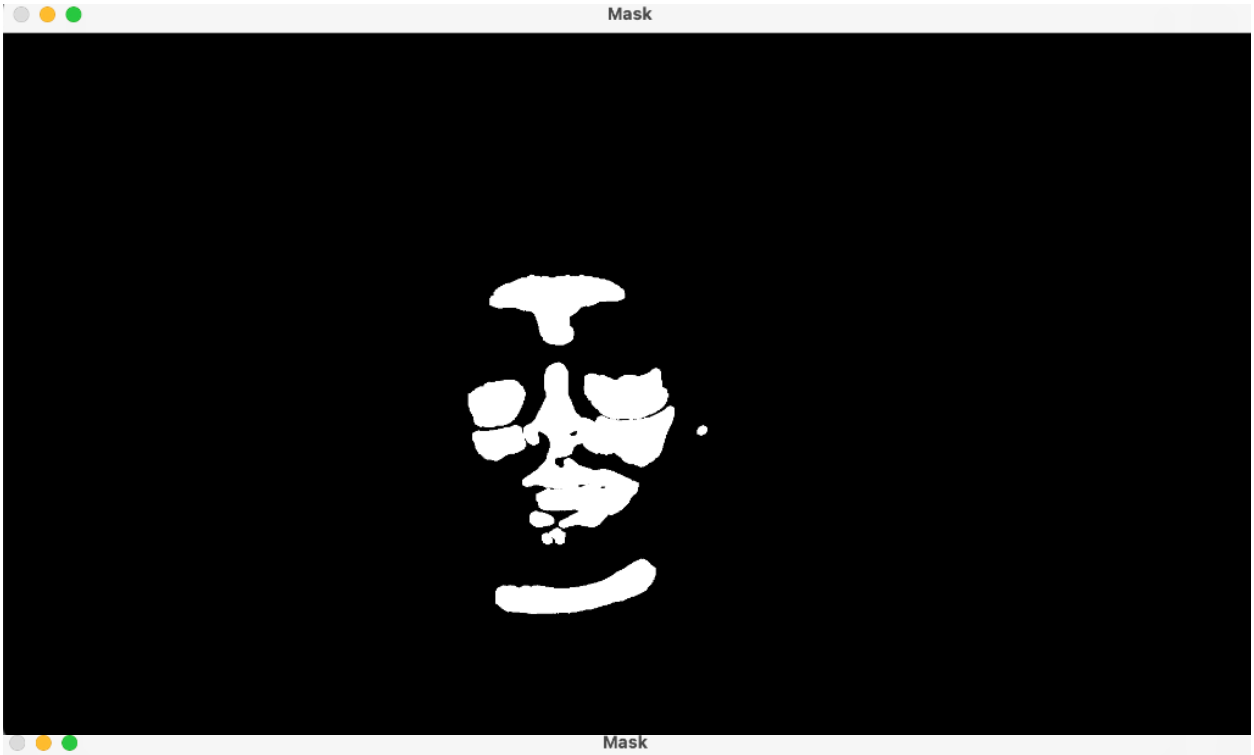
```
def get_biggest_contour_centroid(mask):
    """
    Finds the largest contour in the mask and returns its centroid.
    """
    contours, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

    if not contours:
        return None, None

    # Find likely the face/hand (largest area)
    largest_contour = max(contours, key=cv2.contourArea)

    # Calculate Moments
    M = cv2.moments(largest_contour)
    if M["m00"] == 0:
        return None, None

    cx = int(M["m10"] / M["m00"])
    cy = int(M["m01"] / M["m00"])

    return (cx, cy), largest_contour
```

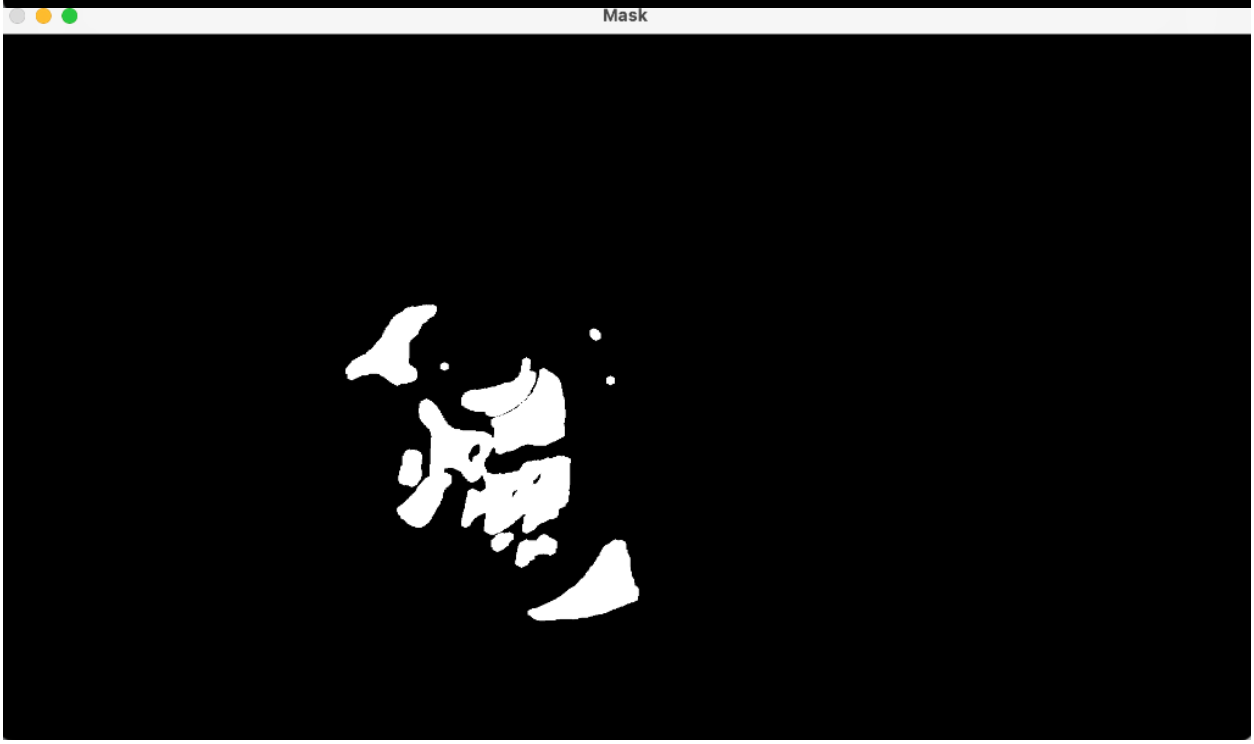
The mouse movement logic using `pyautogui`:

```
# Calculate target (Relative to neutral position)
dx = cx - neutral_x
dy = cy - neutral_y

target_x = center_screen_x + (dx * SCALE_X)
target_y = center_screen_y + (dy * SCALE_Y)

# Smoothing
curr_mouse_x = prev_mouse_x + (target_x - prev_mouse_x) * SMOOTHING_FACTOR
curr_mouse_y = prev_mouse_y + (target_y - prev_mouse_y) * SMOOTHING_FACTOR

pyautogui.moveTo(curr_mouse_x, curr_mouse_y)
```

4. Results

The application successfully captures video, detects skin regions, and moves the mouse. Below are examples of the binary mask generated during operation, showing the segmentation of the face/hand.





5. Conclusion

The implementation demonstrates how computer vision techniques can be used for Human-Computer Interaction. The combination of color thresholding and morphological operations provides a robust mask, while the centroid calculation allows for precise tracking. The addition of smoothing and calibration significantly improves the user experience, making the control smoother and less tiring.